



**HỌC VIỆN
NGÂN HÀNG**
BANKING ACADEMY OF VIETNAM

Lập trình Dart nâng cao cho Flutter

ThS Nguyễn Hoàng Anh



Những nội dung chính bạn sẽ học

- Các khái niệm OOP chuyên sâu được áp dụng trong cơ chế nội tại của Flutter
- Lập trình Generic để thiết kế các thành phần có khả năng tái sử dụng cao
- Cách thức vận hành thực tế của lập trình bất đồng bộ (async) và vòng lặp sự kiện (event loop) trong Dart
- Kiến trúc Stream phía sau Widget StreamBuilder
- Mô hình thiết kế Repository (Repository Pattern) chuẩn bị cho quản lý trạng thái (State Management)



Tính năng ngôn ngữ nâng cao

- Lớp trừu tượng (Abstract classes) & Interface ẩn (implicit interfaces)
- Mixins
- Factory constructors
- Generics

Làm chủ lập trình bất đồng bộ (Async Mastery)

- Hàng chờ Microtask queue vs Event queue
- Chuỗi Future chaining vs async/await
- Streams, broadcast, async*, yield, transformers

Nền tảng kiến trúc ứng dụng

- Tách biệt cấu trúc Model / Repository
- Chuẩn bị kiến thức cho các mô hình BLoC / Riverpod



- Lớp trừu tượng định nghĩa khung cấu trúc mà không cần cài đặt chi tiết tất cả hành vi.
- Ép buộc các lớp con kế thừa phải hiện thực hóa (implement) các phương thức.
- Trong Flutter, `State<T>` là một lớp trừu tượng — các Widget ghi đè (override) các phương thức để định nghĩa giao diện.
- Ví dụ minh họa:

```
abstract class Shape {  
    double area(); // không có thân hàm = bắt buộc phải hiện thực ở lớp con  
}  
  
class Circle extends Shape {  
    final double r;  
    Circle(this.r);  
    double area() => 3.14 * r * r;  
}
```



Khái niệm cơ bản

- Mọi **class** trong Dart đều tự động định nghĩa một interface ngầm định.
- Từ khóa **implements** ép buộc lớp hiện thực phải ghi đè (override) lại toàn bộ thuộc tính và phương thức.

Khi nào nên dùng?

- Định nghĩa các hợp đồng giao tiếp (API contracts)
- Viết Unit Test & Tiêm phụ thuộc (Dependency Injection)
- Tách biệt rõ ràng giữa "lớp cần làm gì" và "lớp làm thế nào"

implicit_interface_example.dart

```
class Bird {  
  void fly() => print("Bird flying");  
}  
  
class Eagle implements Bird {  
  void fly() => print("Eagle gliding");  
}
```

So sánh Extends, Implements & With



Từ khóa	Ý nghĩa	Trường hợp sử dụng
extends	Kế thừa cả định nghĩa và hành vi	Mở rộng hoặc tùy biến từ một lớp có sẵn
implements	Chỉ lấy chữ ký hàm, không nhận hành vi	Định nghĩa hợp đồng (contract), tự cài đặt lại hoàn toàn
with	Nhận/trộn các khối chức năng (Mixin)	Chia sẻ các đoạn mã tái sử dụng giữa nhiều lớp

Ví dụ thực tế trong Flutter:

- StatefulWidget extends Widget
- AnimationController implements Listenable



Khái niệm cơ bản

- Mixin giúp chia sẻ hành vi hoặc mã nguồn mà không tạo ra class

Ứng dụng trong Flutter

- Flutter áp dụng Mixin rất nhiều trong quản lý **Animation**, hỗ trợ

mixin_example.dart

```
mixin Logger {  
  void log(String msg) => print("[LOG] $msg");  
}  
  
class AuthService with Logger {  
  void login() { log("login called"); }  
}
```



Quy tắc hoạt động

- Giới hạn quyền áp dụng Mixin chỉ cho các lớp thuộc một kiểu dữ liệu cha nhất định → đảm bảo các thuộc tính/phương thức cần thiết luôn tồn tại.

Ví dụ minh họa:

Sử dụng từ khóa **on** để định nghĩa ràng buộc kiểu dữ liệu cho Mixin.

mixin_constraints.dart

```
class Pet {  
  final String name;  
  Pet(this.name);  
}  
  
mixin CanBark on Pet {  
  void bark() => print("$name says woof!");  
}  
  
class Dog extends Pet with CanBark {  
  Dog() : super("Buddy");  
}
```



Quy tắc hoạt động

- Kiểm soát việc tạo đối tượng, thay vì luôn bắt buộc tạo ra một

Tối ưu & Ứng dụng

- Tối ưu việc tạo đối tượng, hỗ trợ bộ nhớ đệm (caching), chuyển
- Flutter sử dụng Factory rất nhiều khi định nghĩa Theme, TextS

factory_example.dart

```
class User {  
  final String name;  
  
  factory User.fromJson(Map j) =>  
    User._(j['name']);  
  
  User._(this.name); // private constructor  
}
```

Các trường hợp ứng dụng Factory Constructor



Kịch bản ứng dụng	Ví dụ thực tế
Trả về đối tượng sẵn có	Quản lý dịch vụ duy nhất (Singleton Services)
Chuyển đổi định dạng dữ liệu	Khởi tạo Model từ JSON (.fromJson)
Kiểm tra điều kiện trước khi tạo	Bộ kết nối/thích ứng dữ liệu (Database Adapters)
Khởi tạo bất đồng bộ	Tải tài nguyên (asset) trước khi tạo đối tượng hoàn chỉnh



- Generics = Mẫu định kiểu dữ liệu (templates) — viết mã một lần, tái sử dụng an toàn.

Cụ thể trong Dart: `Future<T>`, `Stream<T>`, `List<T>`, `Map<K, V>`

- Ví dụ minh họa:

```
List<String> students = ["Anna", "Ben"];
```

```
Map<String, int> scores = {"Anna": 95};
```

- Tầm quan trọng:

Đảm bảo an toàn kiểu dữ liệu ngay từ lúc biên dịch (Compile-time type safety)

Tránh việc lạm dụng kiểu dynamic nguy hiểm

Được sử dụng khắp nơi trong Flutter (Widget, Async, Form)



Ví dụ về Lớp Generic (Generic Class)

ĐẶC ĐIỂM

Tối ưu hóa & Tái sử dụng

- Một lớp định nghĩa duy nhất → làm việc an toàn với nhiều kiểu dữ liệu khác nhau.
- Tương tự danh sách `Dropdown<String>` hoặc danh sách chứa Widget `List<Widget>` trong Flutter.

VÍ DỤ MINH HỌA

```
class Box<T> { T value; Box(this.value); void  
show() ⇒ print("Value = $value"); } void main() ⇒  
Box<int>(10).show();
```

// Kết quả màn hình: Value = 10



Ràng buộc cho Generics (Generic Constraints - 'extends')

ĐẶC ĐIỂM

Kiểm soát kiểu dữ liệu

- Bắt buộc kiểu dữ liệu Generic phải là lớp con của một kiểu cụ thể chỉ định trước qua từ khóa **extends**.
- Ứng dụng vô cùng phổ biến trong hệ thống **Animation** và các mô hình **Provider patterns** của Flutter.

VÍ DỤ MINH HỌA

```
abstract class Animal { String sound(); }  
class Cat extends Animal {  
  sound() ⇒ "Meow";  
}  
  
class Box<T extends Animal> {  
  T pet;  
  Box(this.pet);  
  void play() ⇒ print(pet.sound());  
}
```

Cú pháp Collection-if, Collection-for & Toán tử Spread (...)



ĐẶC ĐIỂM

Tối ưu hóa & Tái sử dụng

- Dart cung cấp các cú pháp dựng tập hợp linh hoạt theo phong cách lập trình chức năng.
- Cực kỳ quan trọng khi xây dựng danh sách Widget trong Flutter tại thuộc tính `children:` `[ ... widgets ]`.

VÍ DỤ MINH HỌA

```
var base = [1, 2, 3];
var list = [
  ...base, // toán tử trải spread
  if (true) 99, // chèn phần tử theo điều kiện
  for (var x in base) x * 10 // chèn phần tử thông
  qua vòng lặp
];
print(list);

// Kết quả in ra: [1, 2, 3, 99, 10, 20, 30]
```



Ngoại lệ tự định nghĩa (Custom Exceptions)

MỤC ĐÍCH

Truyền tải ngữ cảnh lỗi nghiệp vụ

- Truyền tải ngữ cảnh lỗi cụ thể theo từng nghiệp vụ (thay vì dùng thông báo văn bản thô).
- Chủ động ném ngoại lệ rõ ràng ở tầng **Repository** → giúp tầng giao diện (UI) dễ dàng bắt lỗi và xử lý logic tương ứng.

VÍ DỤ MINH HỌA

```
class LoginException implements Exception {  
    final String msg;  
    LoginException(this.msg);  
}
```

Vòng lặp sự kiện (Event Loop) & Hàng chờ Microtask



- Nắm vững thứ tự thực thi bất đồng bộ → giúp giao diện UI phản hồi mượt mà hơn.
- Engine xử lý bất đồng bộ của Dart sử dụng 2 hàng chờ (queue):

Hàng chờ	Mức ưu tiên	Chức năng xử lý
Microtask queue	▲ Cao nhất	Các tác vụ xử lý nội bộ tức thì
Event queue	Bình thường	Futures, sự kiện tương tác UI, bộ đếm Timers

- Các tác vụ trong Microtask queue luôn được ưu tiên xử lý xong trước Event queue.



KHÁI NIỆM

Hiểu rõ hàng chờ (Queue)

- Giúp tối ưu hóa để có các animation mượt mà.
- Kiểm soát luồng bất đồng bộ (predictable async) hoạt động một cách chính xác.

VÍ DỤ MINH HỌA

```
import 'dart:async';

void main() {
  print("A");
  scheduleMicrotask(() ⇒ print("micro"));
  Future(() ⇒ print("future"));
  print("B");
}
```

Kết quả in ra □ A, B, micro, future



Chuỗi nối Future (Future Chaining)

KHÁI NIỆM

Chuỗi liên tiếp (Chaining)

- Thực thi các bước bất đồng bộ liên tiếp dạng chuỗi mà không cần lồng mã quá sâu.
- Phù hợp khi không cần khai báo biến tạm hoặc không muốn dùng khối `async/await`.

VÍ DỤ MINH HỌA

```
Future((() => 1)
    .then((v) => v + 1)
    .then((v) => print(v)));
```

Kết quả in ra □ 2



Tạo Stream — `async*`, `yield`, `StreamController` & Broadcast

KHÁI NIỆM

Cơ chế hoạt động Stream

- **`async*` & `yield`:** Hàm tạo Stream bất đồng bộ, sinh chuỗi giá trị liên tiếp theo yêu cầu.
- **`StreamController`:** Đối tượng điều khiển luồng dữ liệu, cho phép thêm phần tử chủ động qua `add()`.
- **Broadcast Stream:** Cho phép nhiều đối tượng lắng nghe (`listen`) đồng thời cùng một luồng sự kiện.

VÍ DỤ MINH HỌA

// 1. Hàm tạo Stream (`async*` & `yield`)

```
Stream<int> nums() async* {  
  for (int i = 1; i <= 3; i++) yield i;  
}
```

// 2. `StreamController` & Broadcast

```
var c = StreamController.broadcast();  
c.stream.listen((v) => print("A:$v"));  
c.stream.listen((v) => print("B:$v"));  
c..add(1)..add(2);
```

Kết quả in ra ☐

A:1 B:1

A:2 B:2



KHÁI NIỆM

Cơ chế Repository

Tách biệt hoàn toàn tầng truy xuất dữ liệu khỏi tầng hiển thị giao diện (UI):

- **Giao diện (UI)** □ **Repository** □ **Tầng dữ liệu (API/Cơ sở dữ liệu)**
- Dễ dàng kiểm thử (Testable), bảo trì tốt, hoạt động mượt mà với Bloc, Riverpod, Provider.

VÍ DỤ MINH HỌA

// Ví dụ sử dụng Future:

```
class Repo {  
  Future<String> getUser() async {  
    await Future.delayed(  
      Duration(milliseconds: 300)  
    );  
    return "Anna";  
  }  
}
```

□ **Tầng UI gọi thực thi:**

```
await repo.getUser()
```



Ví dụ Repository Pattern với Stream

KHÁI NIỆM

Cơ chế hoạt động Stream

- **Kết nối Widget:** Sử dụng `StreamBuilder` để tự động lắng nghe luồng dữ liệu `repo.counter()` và cập nhật UI mỗi khi phát giá trị mới.
- **Nền tảng Real-time:** Giải pháp cốt lõi để phát triển các ứng dụng thời gian thực như hệ thống chat, nhận thông báo, và đồng bộ dữ liệu liên tục.

VÍ DỤ MINH HỌA

// 1. Ví dụ triển khai luồng phát dữ liệu (async* & yield)

```
Stream<int> counter() async* {  
  for (int i = 1; i ≤ 3; i++) {  
    await Future.delayed(Duration(milliseconds: 300));  
    yield i;  
  }  
}
```

// 2. Kết nối với Widget giao diện trong Flutter:

```
StreamBuilder(  
  stream: repo.counter(),  
  builder: ...  
)
```



Tổng hợp — Quy tắc tối ưu hiệu năng ứng dụng

KIẾN TRÚC

Luồng kiến trúc dữ liệu chuẩn

Model



Repository



Stream / Future



Tải / Dựng lại Widget
(Rebuild UI)

HIỆU NĂNG

Quy tắc tối ưu hiệu năng

- **Tránh chặn Isolate chính:** Không chạy các vòng lặp tính toán quá nặng trên luồng UI.
- **Sử dụng từ khóa `const`:** Cho Widget để giảm chi phí dựng lại giao diện.
- **Sử dụng hàm `async`:** Cho các thao tác I/O (mạng, đọc file, bộ nhớ).
- **Isolate riêng cho tác vụ CPU nặng:** Sẽ học ở module sau.



MỤC TIÊU

Xây dựng mô phỏng tầng dữ liệu (Data Layer) bằng Dart

Khai báo Model:

- Tạo lớp mô hình dữ liệu Product (Product Model).

In kết quả thực thi:

- Gọi và hiển thị kết quả thực thi ra màn hình console.

YÊU CẦU REPOSITORY

Xây dựng Repository cung cấp các phương thức:

Future<List<Product>>

- (lấy danh sách sản phẩm)

Stream<Product>

- (phát dữ liệu sản phẩm mới thời gian thực)



NỀN TẢNG

Cấu trúc & Đối tượng

- **Lập trình hướng đối tượng nâng cao** (Advanced OOP)
- **Kỹ thuật Generics**
- **Cú pháp xử lý Tập hợp nâng cao** (Collections)

XỬ LÝ DỮ LIỆU

Luồng & Thiết kế

- **Quản lý và xử lý Ngoại lệ** (Exceptions)
- **Lập trình Bất đồng bộ & Dòng dữ liệu** (Async & Streams)
- **Mô hình thiết kế Repository** (Repo Pattern)



- **Sách Mastering Flutter (2025)**, Chương 2, 3, 6
- Trang tài liệu chính thức: docs.flutter.dev
- Môi trường thực hành: dart.dev/tools/dartpad